🧭 Overview

AI Asset Registries Overview

Purpose

This document defines the product requirements and expected behavior for a unified AI Asset Registry solution intended to support Red Hat OpenShift AI and related platform capabilities. The goal is to establish a scalable registry framework that can govern multiple types of AI assets through a common experience, while avoiding a future in which each new asset type requires a separate bespoke registry implementation.

Problem Statement

AI platforms are rapidly expanding beyond models alone. Organizations increasingly need to manage and govern a growing set of AI assets such as agents, MCP servers, prompts, guardrails, skills, and knowledge sources. Current approaches often create isolated registries one asset type at a time, which can lead to duplicated effort, inconsistent user experiences, fragmented governance, and slower support for new asset types.

This proposal defines a product direction for a registry solution that treats AI assets as first-class citizens and supports a plugin-based approach for adding new registry types over time. The registries are intended to be metadata-first, with the actual assets stored or running elsewhere, while the registry provides governance, lifecycle, policy linkage, relationships, auditability, and integration with the broader platform ecosystem. This aligns with MLflow's metadata/artifact separation and with Kubeflow/OpenShift AI's distinction between model registries and discoverability-oriented catalogs.

Product Vision

The proposed product vision is a federated AI Asset Registry framework with:

- Multiple backend registries where needed
- A unified interface for interacting with registered AI assets
- A plugin model that enables rapid addition of new AI asset types
- Consistent governance, lifecycle, and policy support across those assets
- Support for development and production registry use cases
- Alignment with OpenShift AI, Kubernetes, and gateway-centered platform architecture

The long-term intent is not to force all assets into a single storage mechanism, but to provide a common registry experience and governance model across heterogeneous AI assets and backend systems. This direction is especially relevant as MLflow continues to expand its registry and GenAI capabilities, including Prompt Registry and metadata-first support for externally stored models and agents.

Key Conceptual Boundary

A central concept in this proposal is the distinction between **registries** and **catalogs**.

Registries are intended for governance and lifecycle management. They focus on:

- identity and versioning
- metadata and references
- lifecycle state
- policy linkage
- lineage and relationships
- approvals and promotion
- auditability and observability

Catalogs are intended for discoverability and consumption. They focus on:

- browsing
- searching
- filtering
- curation
- helping users find appropriate assets

This distinction is consistent with how Kubeflow and OpenShift AI describe model catalog and model registry responsibilities today.

Scope of This Document

This document is a **product requirements document**. It defines the product capabilities, requirements, expected behaviors, and future extensibility goals for the proposed registry solution.

This document does **not** prescribe a specific implementation design, internal architecture, storage model, or detailed engineering solution. Those decisions are expected to be evaluated and defined by engineering during architecture and design.

Strategic Importance

This registry capability is expected to become foundational for broader AI platform needs, including:

- evaluation
- observability
- gateway integration
- lifecycle automation
- policy enforcement
- supply chain management
- secure promotion from inner loop to outer loop workflows

Because of that, the registry framework should be designed from the start to support extensibility, security, and integration with future platform capabilities, rather than only solving for a single asset type or near-term use case. Gateway-policy attachment patterns in Gateway API and Kuadrant, along with separation between policy decision and runtime enforcement, reinforce the importance of this foundation.

🏁 Goals and Desired Outcomes

Goals and Desired Outcomes

The desired end state is a product framework that allows Red Hat to govern a broad and growing set of AI assets through a unified, extensible registry experience, while maintaining separation from catalogs, supporting secure platform integration, and enabling future platform growth without repeated bespoke registry efforts.

Goals

The proposed AI Asset Registry solution should provide a scalable and extensible product foundation for governing AI assets across Red Hat OpenShift AI and related platform capabilities. The primary goals are:

Enable AI assets to be treated as first-class citizens
The product should support governance and lifecycle management for a growing set of AI assets beyond models alone, including agents, MCP servers, prompts, guardrails, knowledge sources, and skills.

Avoid one-off registry development for each new asset type
The product should establish a common registry framework that can support new AI asset types through a plugin-based approach, reducing duplicated effort and improving long-term maintainability.

Provide a unified registry experience across multiple backend registries
The product should present a consistent and cohesive experience for users and administrators even when different asset types rely on different backend registry technologies.

Support metadata-first governance
The product should manage metadata, references, relationships, lifecycle state, policy linkage, and governance context, while allowing the actual assets and running services to exist elsewhere.

Distinguish governance from discoverability
The product should clearly separate the role of registries from the role of catalogs, while allowing them to work together seamlessly.

Support development and production usage patterns

The product should support both development-oriented registries and production-oriented registries, including promotion and lineage across environments.

Enable inner-loop to outer-loop workflows
The product should make it easier for users to move from local development to cluster-based development and eventually to production-ready environments with appropriate governance and policy enforcement.

Align with OpenShift AI and OpenShift-native platform patterns
The product should fit naturally into the Red Hat OpenShift AI ecosystem and account for Kubernetes, CRDs, APIs, containers, multi-tenancy, and other platform expectations where appropriate.

Create a foundation for future platform capabilities
The product should serve as an enabling layer for related capabilities such as evaluation, observability, gateway integration, lifecycle automation, policy enforcement, and supply chain management.

Preserve flexibility for future growth
The product should make it easier to add new AI asset types, new policy needs, new integration patterns, and future ecosystem requirements without requiring major redesign.

Desired Outcomes

If successful, this product opportunity should lead to the following outcomes.

Faster support for new AI asset types
New asset types should be added through a predictable extension model rather than requiring a brand-new registry effort each time.

A more consistent user and admin experience
Users, platform engineers, and administrators should experience a more unified way of working with AI assets, even when the backend systems differ.

Improved governance across heterogeneous AI assets

Organizations should be able to apply common governance expectations across multiple AI asset types, including lifecycle tracking, auditability, access control, lineage, and policy linkage.

Better alignment between platform capabilities
Registries should become a stronger foundation for other strategic capabilities such as gateways, evaluations, observability, lifecycle operators, and policy enforcement.

Reduced fragmentation across the platform
The product should reduce the risk of isolated solutions being built independently for each asset type, which can create duplication, inconsistency, and integration challenges.

Better support for secure enterprise adoption
The product should help enterprises manage AI assets with stronger support for security, tenancy, policy-aware workflows, promotion controls, and integration with identity and access systems.

A clear path from development to production
Users should have a more understandable and governed path for moving assets from local or development contexts into shared cluster environments and production use cases.

Stronger long-term product positioning
Red Hat should be better positioned to support a broader AI asset ecosystem over time, rather than only addressing isolated registry needs one at a time.

A framework that supports both current and future priorities
The product should support the current priority assets while also leaving room for future assets, partner ecosystem onboarding, and downstream productization needs.

Scope

Scope

This proposal defines the product scope for a unified AI Asset Registry solution that supports governance and lifecycle management for multiple AI asset types through a common product framework.

The solution is intended to provide a consistent registry experience across multiple backend registries while supporting extensibility for additional AI asset types over time. The registry solution is expected to be metadata-first, with actual assets, files, models, services, or runtime components stored and operated elsewhere.

This proposal is focused on product capabilities and expected behavior. It defines what the product should support and how it should conceptually operate within the broader Red Hat OpenShift AI ecosystem.

In Scope

The following areas are in scope for this proposal:

- **Unified registry framework**
  A common product framework for working with multiple AI asset registries through a unified experience.
- **Metadata-first registry behavior**
  Support for managing metadata, references, relationships, lifecycle context, policy linkage, auditability, and governance state rather than requiring the registry to store the actual asset payloads.
- **Multiple AI asset types**
  Support for current priority asset types, with the expectation that additional asset types can be added over time through a plugin-based model.
- **Plugin-based extensibility**
  A product expectation that new AI asset types can be introduced without requiring a full redesign or bespoke registry effort for each new type.
- **Multiple backend registries**
  Support for a federated approach in which different asset types may rely on different backend registry systems while still participating in a unified product experience.

- **Development and production registry support**
  Support for development-oriented and production-oriented registry usage patterns, including promotion, lineage, and governance expectations across environments.
- **Inner-loop to outer-loop workflows**
  Support for local development, development on cluster, and progression toward governed shared environments and production use cases.
- **OpenShift AI alignment**
  Alignment with Red Hat OpenShift AI, Kubernetes, APIs, CRDs, containers, tenancy, security expectations, and related platform patterns where appropriate.
- **Catalog and registry interoperability**
  Support for clear separation between registries and catalogs while allowing them to work together in a coherent user experience.
- **Security and policy-aware governance**
  Support for access control, identity-aware workflows, policy association, auditability, and alignment with gateway-based enforcement models.
- **Future platform enablement**
  Support for broader platform capabilities that depend on governed AI assets, such as evaluation, observability, lifecycle automation, gateway integration, and supply chain controls.

Out of Scope

The following areas are out of scope for this document:

- **Detailed implementation design**
  This document does not define the internal engineering design, controller design, storage implementation, or specific service architecture.
- **Backend-specific implementation decisions**
  This document does not prescribe exactly how MLflow, Kubeflow, or any other backend must be extended or implemented.
- **Final API schema design**
  This document does not define final API payloads, CRD schemas, or detailed developer interface specifications.

- **Detailed UI design**
  This document does not define page layouts, navigation, wireframes, or full user experience design.

- **Full lifecycle automation for every asset type in MVP**
  Lifecycle automation is an important direction, but this document does not assume that every asset type must have full automation in the first version.

- **Full supply chain feature completeness in MVP**
  Supply chain, attestation, provenance, and related controls are strategically important, but they are not assumed to be fully implemented for every asset type in the initial scope.

- **Mandatory support for every possible AI asset type on day one**
  The framework should be extensible, but the initial product scope is expected to focus on the priority asset types first.

📘 Core Concepts

Core Concepts

This section defines the core concepts that shape the proposed AI Asset Registry product direction. These concepts are intended to establish a shared understanding across Product Management, Engineering, and Leadership so that the opportunity, scope, and expected behavior of the solution are interpreted consistently. The definitions below are provided at the product level to clarify how key terms such as registries, catalogs, plugins, AI assets, governance, and environment flow are meant to be understood within this proposal.

AI Asset

An AI Asset is a governed unit within the platform that represents an important component of an AI solution, workflow, runtime, or supporting capability.

Examples include agents, MCP servers, prompts, models, guardrails, knowledge sources, and skills.

An AI asset may be internally created, externally sourced, deployed as a service, referenced from another system, or used as part of a larger composed solution.

Registry

A Registry is the product capability responsible for governance and lifecycle management of AI assets.

A registry is intended to manage metadata and references related to an asset, including identity, versioning, lifecycle state, governance context, access expectations, relationships, policy linkage, and auditability.

A registry is not primarily defined by storing the actual asset payload. In most cases, the asset itself may live elsewhere while the registry governs the metadata and context needed to manage and use it responsibly.

Catalog

A Catalog is the product capability responsible for discoverability and consumption-oriented access to assets.

A catalog helps users browse, search, filter, evaluate, and select appropriate assets. A catalog may surface curated or approved assets, but it is conceptually distinct from the registry function.

In this proposal, registries and catalogs are separate but complementary product capabilities.

Plugin

A Plugin is the conceptual extension mechanism that allows the registry framework to support new AI asset types over time.

A plugin should allow an asset type to define its own metadata needs, configuration expectations, relationships, lifecycle considerations, and governance requirements while still participating in the shared registry framework and unified product experience.

The goal of the plugin concept is to avoid repeated one-off registry creation for every new asset type.

Unified Interface

The Unified Interface is the expected common experience for interacting with multiple AI asset registries.

This may include APIs, Kubernetes-native interfaces such as CRDs, UI experiences, CLI capabilities, or other interaction models. The key concept is that users and platform teams should encounter a consistent product experience even when the underlying registries differ.

Federated Registry Model

The Federated Registry Model refers to an architecture in which multiple backend registries may exist, but the product presents them through a common framework and experience.

This allows different asset types to use backend systems that are appropriate for their current needs while still participating in a broader governance and lifecycle model.

Metadata-First Governance

Metadata-First Governance means that the registry focuses on the descriptive, operational, and governance metadata needed to manage AI assets rather than assuming the registry must contain the asset itself.

This includes metadata such as:

- identity
- version
- owner
- source
- endpoint or location reference
- lifecycle state
- policy association
- access model
- relationships to other assets
- audit and observability context

Dev Registry

A Dev Registry is a registry context intended to support AI engineers and development workflows.

It supports active development, experimentation, iterative updates, local-to-cluster workflows, and earlier lifecycle stages, while still operating within platform governance expectations.

Prod Registry

A Prod Registry is a registry context intended to support governed production workloads used in business or operational contexts.

It is expected to have stronger requirements around promotion, approval, immutability expectations, governance, security posture, and policy enforcement.

Inner Loop to Outer Loop

Inner Loop to Outer Loop describes the expected progression from local or early development workflows into shared, governed, and production-ready environments.

In this proposal, the registry framework should support this progression by enabling asset portability, promotion, lineage tracking, and smoother transitions between development and production contexts.

Policy Linkage

Policy Linkage refers to the association of an AI asset with the policies, controls, or governance expectations that apply to it.

The registry is expected to carry the metadata or references needed to express those associations, while enforcement may occur through other platform layers such as gateways, policy engines, controllers, or operators.

Gateway Enforcement

Gateway Enforcement refers to the application of runtime controls through gateway technologies that mediate access to AI-related services or traffic.

Examples may include inference gateways, MCP gateways, semantic routing layers, and future agent gateways. In this proposal, the registry should support integration with gateway-centered enforcement models by carrying the metadata and policy context needed for governance and runtime control alignment.

Relationship Model

The Relationship Model refers to the ability to express meaningful connections between AI assets.

Examples may include an agent referencing a model, a prompt, an MCP server, a guardrail, or a knowledge source. These relationships are important for governance, lifecycle awareness, policy application, and future platform capabilities.

First-Class AI Assets

Treating AI assets as first-class citizens means recognizing that important AI building blocks beyond models also require formal governance, lifecycle management, discoverability support, and platform integration.

This concept is central to the proposal and helps justify the need for a broader registry framework rather than isolated point solutions.

👥 Target Users and Personas

Target Users and Personas

This section describes the primary users and stakeholder groups for the proposed AI Asset Registry solution. These personas help clarify who the product is intended to serve, how each group interacts with it, and what value they should gain from it. While not every persona will use the product directly in the same way, each plays an important role in the broader lifecycle of governed AI assets.

AI Engineers

AI Engineers are primary users of the registry ecosystem. They develop AI applications and workflows, consume governed AI assets, and in some cases create or contribute new assets such as prompts, agents, skills, or knowledge-source integrations.

Their main interests include:

- finding approved and usable AI assets
- understanding asset metadata, status, and compatibility
- referencing governed assets during development
- promoting assets through development workflows
- understanding relationships between assets such as agents, prompts, models, guardrails, and MCP servers

For this group, the registry should make AI assets easier to understand, trust, reuse, and move through development and promotion workflows.

Platform Engineers

Platform Engineers are responsible for setting up, integrating, and maintaining the underlying AI platform environment. They are less focused on day-to-day asset authoring and more focused on ensuring the platform can support registry capabilities securely and reliably.

Their main interests include:

- enabling registry capabilities across environments
- integrating registries with OpenShift AI and other platform services
- supporting tenancy, access control, and operational consistency

- ensuring the registry framework fits with APIs, Kubernetes-native interfaces, gateways, and related infrastructure
- maintaining a scalable foundation for current and future asset types

For this group, the product should provide a clear and extensible framework that can be supported operationally across development and production environments.

AgentOps Admins

AgentOps Admins are responsible for governance, approval, enablement, and operational oversight of AI assets within the platform. They play a key role in ensuring that assets are properly registered, reviewed, controlled, and made available for safe enterprise use.

Their main interests include:

- reviewing and approving AI assets
- controlling which assets are enabled for use
- managing governance state and lifecycle progression
- associating assets with the right policies and controls
- understanding asset lineage, ownership, and environment status
- supporting promotion from development to more governed runtime contexts

For this group, the registry should provide strong governance capabilities, policy-aware workflows, and a clear operational view of managed AI assets.

Business Consumers

Business Consumers are generally indirect users of the registry solution. They are not expected to manage registry data directly, but they benefit from the governed assets that become available through applications, assistants, services, and curated catalogs.

Their main interests include:

- access to trusted and approved AI-powered capabilities
- consistent and reliable user experiences
- lower risk from ungoverned or unsupported AI assets
- confidence that business-facing AI solutions are backed by proper governance and lifecycle controls

For this group, the value of the registry is mostly indirect: it helps create a safer, more manageable, and more scalable foundation for the AI capabilities they ultimately use.

Product Managers and Leadership

Product Managers and Leadership are strategic stakeholders in the registry solution. They are responsible for defining direction, prioritizing capabilities, aligning teams, and ensuring the product supports broader business and platform goals.

Their main interests include:

- reducing fragmentation across AI asset management
- enabling a scalable product direction rather than repeated one-off efforts
- supporting current and future AI asset types
- aligning registry capabilities with platform strategy, governance, and ecosystem growth
- creating a foundation for future capabilities such as lifecycle automation, evaluation, observability, and gateway integration

For this group, the product should represent a strong strategic foundation that can grow with platform needs and support long-term product differentiation.

⚒️ Jobs to be Done

Jobs to Be Done (JTBD)

This section defines the key jobs the proposed AI Asset Registry solution must enable across the lifecycle of governed AI assets. Unlike personas, these jobs are not tied to a single role or title. In practice, different organizations may assign these responsibilities to different people or teams. The purpose of this section is to clarify the core work the system must support so that product requirements remain focused on capability and outcome rather than on rigid role definitions.

Discover and Understand Available AI Assets

Users need to identify what AI assets are available, understand what they are, determine whether they are appropriate for use, and see the context needed to work with them responsibly.

The system should support:

- visibility into available governed assets
- clear metadata and descriptive context
- understanding of ownership, status, and version
- visibility into relationships with other assets
- enough context to support informed selection and use

Register and Describe AI Assets

Users need to bring AI assets under governance by registering them and defining the metadata, references, and contextual information needed to manage them within the platform.

The system should support:

- registering different AI asset types
- associating assets with the correct metadata
- referencing assets that live or run elsewhere
- capturing asset-specific configuration needs
- supporting extensibility for additional asset types over time

Govern AI Assets Consistently

Users need to apply governance to AI assets in a way that is consistent, scalable, and adaptable across different asset types and environments.

The system should support:

- common governance expectations across asset types
- lifecycle visibility and status tracking
- policy linkage and governance context
- review and approval workflows where needed
- auditability and controlled change over time

Reuse Trusted AI Assets

Users need to confidently reuse approved and governed AI assets rather than recreating similar assets repeatedly or relying on unmanaged components.

The system should support:

- reuse of trusted assets across teams and projects
- clear signals of approval, readiness, and intended usage
- easier referencing of existing assets during development
- reduced duplication of asset creation effort
- better consistency across AI solutions

Understand Relationships Between AI Assets

Users need to understand how AI assets relate to one another so they can safely build, govern, troubleshoot, and evolve larger AI solutions.

The system should support:

- visibility into asset-to-asset relationships
- awareness of dependencies and references
- understanding of how one governed asset affects another
- support for composed solutions that involve multiple asset types
- future platform capabilities that depend on relationship awareness

Promote AI Assets Across Environments

Users need to move AI assets from development-oriented contexts into more governed and production-oriented contexts while preserving lineage, trust, and control.

The system should support:

- promotion from local or dev workflows toward shared environments
- progression from development registries to production registries
- lineage visibility across environments
- governance-aware transition between lifecycle stages
- consistency between inner-loop and outer-loop workflows

Enable Policy-Aware Usage of AI Assets

Users need AI assets to carry the governance and policy context required for responsible use, while allowing enforcement to occur through the appropriate platform layers.

The system should support:

- association of assets with applicable policies and controls
- visibility into what governance expectations apply
- alignment between registry metadata and enforcement systems
- policy-aware usage across development and production contexts
- a strong foundation for future enforcement and compliance needs

Support Secure Access and Controlled Enablement

Users need to ensure that AI assets are visible, usable, and manageable only by the appropriate users, systems, and environments.

The system should support:

- secure access patterns
- role- and policy-aware visibility
- controlled enablement of assets
- integration with identity and access expectations
- support for enterprise governance and operational control

Integrate AI Assets with the Broader Platform

Users need governed AI assets to work cleanly with the broader AI platform, including discoverability, gateways, operations, and future automation capabilities.

The system should support:

- interoperability with catalogs
- alignment with OpenShift AI and Kubernetes-native patterns
- integration with gateway-centered platform design
- support for observability, evaluation, and lifecycle needs
- compatibility with current and future platform services

Extend the Registry Framework as New Asset Types Emerge

Users need the platform to support new AI asset types without requiring repeated one-off registry efforts or major redesign.

The system should support:

- simple addition of new asset types
- a plugin-based extension model
- consistency between current and future asset registries
- flexibility for varying asset-specific metadata and configuration needs
- long-term scalability of the product direction

Summary

The core job of the proposed AI Asset Registry solution is to make AI assets governable, understandable, reusable, promotable, and extensible across the platform. By focusing on these jobs rather than on fixed roles, the product can better support the reality that different organizations distribute responsibilities differently while still requiring the same underlying capabilities from the system.

AI Asset Coverage

AI Asset Coverage

This section defines the AI asset types the proposed registry framework is expected to support. It identifies the current priority assets that should shape the initial product direction, along with additional asset types that may become important over time. The purpose of this section is to clarify the expected breadth of the product opportunity and reinforce the need for a scalable, plugin-based registry framework rather than isolated registry solutions built one asset type at a time.

The asset types listed below are not all expected to carry identical metadata, lifecycle, or governance needs. Each asset type may require its own configuration, relationships, controls, and operational context. The registry framework should therefore support both consistency across asset types and flexibility for asset-specific requirements.

Primary AI Assets

The following asset types represent the current priority scope for registry support.

Agents

Agents are a primary AI asset type and should be treated as first-class governed assets within the registry framework.

Agents often serve as composed runtime capabilities that may reference other governed assets such as models, prompts, MCP servers, skills, guardrails, and knowledge sources. Because of this, agent registry support is important not only for governing the agents themselves, but also for understanding and governing the relationships between multiple AI assets that work together.

At a product level, the registry should support the ability to identify, describe, govern, version, and track agents as distinct AI assets.

MCP Servers

MCP Servers are a high-priority asset type due to their growing importance as governed runtime integrations and tool-serving capabilities within the broader AI ecosystem.

MCP servers may be internally developed, partner-provided, or third-party sourced. They may also require strong governance around enablement, connectivity, security, policy association, lifecycle state, and gateway alignment.

At a product level, the registry should support the ability to identify, describe, govern, and operationally reference MCP servers as managed AI assets, including their relationship to runtime services and platform controls.

Prompts

Prompts are a key AI asset type and should be governable as reusable platform assets rather than treated only as embedded implementation details.

Prompts may be created internally, reused across use cases, versioned over time, and associated with agents, workflows, or other solution components. While prompt support already exists in MLflow, the long-term product direction should account for prompts as part of the broader plugin-based registry framework rather than as a special isolated case.

At a product level, the registry should support the ability to register, version, govern, and reuse prompts as first-class AI assets.

Models

Models are foundational AI assets and remain strategically important to the overall registry direction.

In the current environment, model support may rely on existing Kubeflow-oriented capabilities. Over time, there is interest in bringing model registry capabilities into closer alignment with the broader MLflow-based direction so that models can participate more fully in a unified AI asset registry strategy.

At a product level, the framework should account for models as a core asset type, even if model registry support continues to involve separate backend considerations in the near term.

Guardrails

Guardrails are an important AI asset type because they help define the safety, behavioral, or governance boundaries applied to AI-enabled interactions and services.

As guardrail capabilities evolve, they are increasingly likely to require stronger governance, versioning, approval, and relationship tracking. Guardrails may also need to align with runtime enforcement points, gateway patterns, and policy-aware workflows.

At a product level, the registry should support guardrails as governable AI assets with clear metadata, lifecycle state, and integration relevance.

Knowledge Sources

Knowledge Sources are an important asset type because they represent the governed sources of information used by AI systems, agents, retrieval workflows, or related solution components.

These may include structured systems, documents, repositories, connectors, or other referenced information sources. They often carry important governance, trust, access, and lifecycle implications.

At a product level, the registry should support the ability to describe and govern knowledge sources as assets that can be referenced, approved, and managed within a larger AI solution context.

Skills

Skills are a priority asset type because they represent reusable AI-enabled capabilities that may be attached to agents, workflows, or other runtime constructs.

Skills may encapsulate logic, orchestration, or reusable functions that should be governable, discoverable, and manageable apart from a single application or agent definition.

At a product level, the registry should support the ability to identify, describe, govern, and reuse skills as distinct AI assets.

Secondary and Future AI Assets

The following asset types are not the current priority, but they are important contenders for future registry support and should be considered when defining the extensibility needs of the framework.

Notebooks

Notebooks may become important registry candidates as governed development and experimentation assets, especially where they represent reusable work, reference implementations, or controlled workflows.

Pipelines

Pipelines may become important where repeatable AI workflows, data flows, evaluation flows, or operational processes need governance, lifecycle tracking, and reuse across teams or environments.

Evaluators

Evaluators may become increasingly important as AI systems require stronger assessment, validation, benchmarking, and approval workflows. Over time, evaluators may warrant treatment as governed AI assets in their own right.

Skill Packs

Skill Packs may emerge as higher-level grouped capabilities made up of multiple reusable skills or related components. These may require their own packaging and governance treatment beyond individual skills.

Code Snippets

Code Snippets may become useful registry candidates where reusable AI-related logic, examples, or governed implementation fragments need to be shared and managed in a structured way.

Workflows

Workflows may become important as governed composed assets that define how multiple AI components, tools, and controls work together to produce an outcome.

Coverage Expectations

The registry framework should not assume that every asset type behaves the same way. Different asset types may require different:

- metadata
- relationship models
- lifecycle states
- policy needs
- configuration requirements
- runtime references
- governance workflows

For that reason, the product should support a shared registry framework with enough flexibility to accommodate asset-specific needs while still providing a unified overall experience.

The product should also allow organizations to adopt only the asset types they need. Some users may only require governance for a subset of AI assets, while others may want broader adoption over time. The framework should support that variability without requiring redesign.

Summary

The proposed AI Asset Registry solution should support a growing ecosystem of governed AI assets, beginning with the current priority set of agents, MCP servers, prompts, models, guardrails, knowledge sources, and skills. At the same time, it should preserve the flexibility to expand into additional asset types as product needs evolve. This reinforces the need for an extensible registry framework that can support multiple asset types consistently without forcing them into an identical or overly rigid model.

🚀 Product Capabilities

Product Capabilities

This section defines the major product capabilities the proposed AI Asset Registry solution should provide. These capabilities describe what the product must enable at a system level in order to support governed AI assets across multiple asset types, environments, and backend registries. They are intentionally written at the product level and do not prescribe a specific implementation approach.

The capabilities below reflect the need for a metadata-first, plugin-based registry framework that can support current priority AI assets while also creating a scalable foundation for future platform growth.

Unified Registry Experience

The product should provide a unified experience for interacting with AI asset registries, even when different asset types rely on different backend registry technologies.

This capability should allow users and platform teams to work through a consistent product framework rather than navigating completely separate experiences for each asset type. The unified experience may span APIs, Kubernetes-native interfaces, UI experiences, CLI workflows, or other interaction models supported by the platform.

Plugin-Based Asset Extensibility

The product should support a plugin-based extension model for AI asset types.

This capability is intended to allow new registry types to be added over time without requiring a separate bespoke registry effort for each new asset. It should support both consistency across assets and flexibility for asset-specific metadata, relationships, lifecycle needs, and governance expectations.

Metadata-First Asset Management

The product should manage AI assets primarily through metadata, references, and governance context rather than assuming the registry must store the actual asset payload.

This capability should allow the registry to describe and govern assets that are stored, deployed, or operated elsewhere while still maintaining the information needed for lifecycle management, policy linkage, auditability, and platform integration.

Multi-Asset Registry Support

The product should support multiple AI asset types through a shared product framework.

This capability should begin with the priority asset types identified in this proposal, including agents, MCP servers, prompts, models, guardrails, knowledge sources, and skills, while preserving the ability to expand to additional asset types over time.

Asset Registration and Description

The product should allow AI assets to be registered and described in a governed way.

This capability should support the capture of asset identity, ownership, version, source references, runtime references where appropriate, descriptive metadata, and other asset-specific information needed to manage the asset within the broader platform. It should also allow different asset types to carry different configuration and metadata needs.

Asset Versioning and Lineage

The product should support version awareness and lineage tracking for governed AI assets.

This capability should allow users and administrators to understand how assets evolve over time, how versions relate to one another, and how assets progress across development and production-oriented contexts. It should also support preservation of identity lineage as assets are promoted across environments.

Asset Relationship Awareness

The product should support relationship modeling between AI assets.

This capability should allow the platform to represent and expose meaningful relationships such as dependencies, references, associations, or compositions between assets. These relationships are important for understanding how larger AI solutions are assembled and for enabling future capabilities such as lifecycle coordination, evaluation, observability, and policy-aware governance.

Governance and Lifecycle Visibility

The product should provide governance and lifecycle visibility for AI assets across their usable states.

This capability should support clear lifecycle tracking, governance context, approval awareness, and status visibility so that users and administrators can understand whether an asset is draft, reviewed, approved, enabled, promoted, deprecated, or otherwise governed within the platform. The product should allow a common lifecycle approach while preserving flexibility for asset-specific needs.

Policy Linkage

The product should support association between AI assets and the policies, controls, or governance expectations that apply to them.

This capability should allow the registry to carry the metadata, references, or context needed to express those associations without requiring the registry itself to perform all enforcement. It should support policy-aware usage patterns across environments and lay the foundation for tighter integration with enforcement layers over time.

Secure Access and Controlled Enablement

The product should support secure access to AI asset information and controlled enablement of AI assets for use.

This capability should account for enterprise security expectations such as role-aware visibility, controlled asset availability, identity-aware access patterns, and support for broader platform security integration. It should help ensure that assets are visible, manageable, and usable only within the appropriate governance boundaries.

Dev and Prod Registry Support

The product should support both development-oriented and production-oriented registry usage patterns.

This capability should recognize that development registries and production registries may serve different governance needs and different operational expectations, while still participating in a broader shared framework. It should support use cases ranging from active asset development

and experimentation to more tightly governed runtime contexts used by business-facing solutions.

Promotion Across Environments

The product should support movement of governed AI assets across environments.

This capability should enable progression from local development or dev-oriented registry contexts toward more governed shared environments and production registries. It should support promotion-aware workflows, preservation of lineage, and continuity of governance information across environments.

Inner-Loop to Outer-Loop Support

The product should support a smoother path from local or early development workflows into shared, governed, and production-ready platform usage.

This capability should account for the reality that some users may develop locally, some may connect directly to cluster-based development environments, and some may need to promote governed assets from one context to another. The product should make that progression easier rather than forcing disconnected workflows.

Catalog Interoperability

The product should interoperate cleanly with catalogs while preserving the conceptual separation between registry and catalog responsibilities.

This capability should allow registries and catalogs to work together in a coherent user experience without collapsing the two concepts into one. Registries should remain focused on governance and lifecycle, while catalogs remain focused on discoverability and consumption.

OpenShift AI and Platform Alignment

The product should align with Red Hat OpenShift AI and broader OpenShift-native platform expectations.

This capability should support the product's fit within Kubernetes-centered platform patterns, including APIs, CRDs, tenancy, containers where appropriate, and compatibility with the broader operational model expected in OpenShift-based environments.

Gateway-Aware Integration

The product should support integration with gateway-centered platform architecture.

This capability should allow governed AI assets to carry the information needed to align with gateway-mediated access and enforcement patterns, including current and emerging gateway types relevant to AI workloads. This is important for assets such as MCP servers, models, agents, and guardrails, which may have runtime access, routing, or enforcement implications.

Auditability and Observability Context

The product should provide the metadata context needed to support auditing and observability of governed AI assets.

This capability should help platform teams understand who owns an asset, what changed, how it is governed, where it is referenced, and how it relates to broader operational and policy-aware workflows. It should also create a stronger foundation for future integration with observability and operational tooling.

Selective Adoption by Asset Type

The product should allow organizations to adopt the asset types they need without requiring full adoption of every supported registry type.

This capability is important because different organizations may choose to govern only a subset of AI assets at first, while others may expand adoption over time. The framework should support that variability without requiring redesign or all-or-nothing usage.

Future Platform Enablement

The product should create a foundation for future platform capabilities that depend on governed AI assets.

This capability should position the registry framework to support future needs such as evaluation, lifecycle automation, supply chain controls, ecosystem onboarding, observability expansion, and deeper governance integration without requiring the product to be rethought from scratch later.

Summary

The proposed AI Asset Registry solution should provide a unified, extensible set of product capabilities that make AI assets governable, describable, reusable, promotable, and integrable across the platform. These capabilities should support current priority assets and current platform needs while also creating a flexible product foundation for future asset types, future governance models, and future platform growth.

📝 Functional Requirements

Functional Requirements

Introduction

The following functional requirements define the expected product behavior for the proposed AI Asset Registry solution. These requirements are written at the product level and describe what the solution must provide, without prescribing a specific implementation design. The requirements are organized by capability area to make them easier for Product, Engineering, and Leadership to review.

---

1. Unified Registry Framework

- **FR-001** provide a unified registry framework for governed AI assets.
- **FR-002** present AI asset registries as a single cohesive product capability rather than as separate registry products for each asset type.
- **FR-003** support unified interaction through APIs.
- **FR-004** support Kubernetes-native interaction through CRD-based or equivalent platform-native interfaces.
- **FR-005** support a consistent user experience across interfaces used to manage or consume governed AI asset metadata.
- **FR-006** support a unified registry framework that can incorporate or integrate asset-specific registry capabilities without exposing them to users as separate product experiences.

---

2. Metadata-First Registry Behavior

- **FR-007** manage AI assets primarily through metadata, references, and governance context.
- **FR-008** support registration of AI assets whose actual payloads, source artifacts, or running services are mostly stored or operated outside the registry.
- **FR-009** support persistent identity for each governed AI asset.
- **FR-010** support version-aware registration and management of governed AI assets.

- **FR-011** support descriptive metadata for AI assets, including asset-specific metadata as required.
- **FR-012** support references to asset source locations, artifact locations, repository locations, service endpoints, or other externally managed locations as appropriate for the asset type.
- **FR-013** support governance metadata including ownership, lifecycle state, policy association, and audit context.
- **FR-014** not require all AI assets to conform to one identical metadata structure when asset-specific differences are necessary.

---

3. AI Asset Type Coverage

- **FR-015** support governance of multiple AI asset types through the shared registry framework.
- **FR-016** support the following primary AI asset types as priority scope for the product direction: agents, MCP servers, prompts, models, guardrails, knowledge sources, and skills.
- **FR-017** support the future addition of new AI asset types without requiring a redesign of the overall registry framework.
- **FR-018** allow organizations to adopt only the AI asset types relevant to their needs.
- **FR-019** allow different AI asset types to define different metadata, relationships, lifecycle needs, and governance expectations.

---

4. Plugin-Based Extensibility

- **FR-020** support a plugin-based extension model for AI asset types.
- **FR-021** allow new AI asset types to be added through an extension mechanism rather than requiring a bespoke new registry product effort.
- **FR-022** support plugin-defined metadata requirements for asset types.
- **FR-023** support plugin-defined configuration needs for asset types.
- **FR-024** support plugin-defined relationship models and constraints for asset types.
- **FR-025** support plugin-defined lifecycle considerations for asset types.
- **FR-026** support plugin-defined validation needs for asset types.
- **FR-027** support addition of new asset-type plugins with minimal impact to the overall registry framework.

5. Asset Registration and Management

- **FR-028** allow governed AI assets to be registered into the registry framework.
- **FR-029** allow governed AI assets to be updated in accordance with governance and lifecycle rules.
- **FR-030** allow governed AI assets to be versioned.
- **FR-031** allow governed AI assets to be described with asset-specific and common metadata.
- **FR-032** allow governed AI assets to include references to related assets.
- **FR-033** allow governed AI assets to include references to external locations or runtime endpoints where relevant.
- **FR-034** support retrieval and inspection of asset metadata, status, governance context, and relationships.
- **FR-035** support clear differentiation between mutable descriptive information and version-specific governed records where needed by the asset type.

---

6. Relationships, Lineage, and Dependencies

- **FR-036** support explicit relationships between AI assets.
- **FR-037** support relationship types such as dependency, usage, association, protection, evaluation, deployment, promotion lineage, or similar concepts as needed.
- **FR-038** allow relationship rules and constraints to vary by asset type.
- **FR-039** allow users and administrators to view relationships between governed AI assets.
- **FR-040** support lineage tracking across versions of an asset.
- **FR-041** support lineage tracking across environments when an asset is promoted.
- **FR-042** preserve identity lineage across dev and prod contexts.
- **FR-043** support relationship awareness sufficient to enable future platform capabilities such as evaluation, observability, lifecycle coordination, and policy-aware governance.

---

7. Governance and Lifecycle Management

- **FR-044** support governance state for governed AI assets.

- **FR-045** support lifecycle visibility for governed AI assets.
- **FR-046** support a standard lifecycle model across the registry framework.
- **FR-047** allow extension of lifecycle states or rules for specific asset types where needed.
- **FR-048** support review, approval, enablement, deprecation, and similar governance-oriented status concepts as required by the asset type or environment.
- **FR-049** allow stricter governance expectations to be applied in production-oriented contexts than in development-oriented contexts.
- **FR-050** support clear visibility into whether an asset is draft, governed, approved, enabled, promoted, deprecated, retired, or otherwise governed according to the applicable lifecycle model.
- **FR-051** support governance continuity as assets move between environments.

---

8. Development, Promotion, and Environment Progression

- **FR-052** support development-oriented registry usage patterns.
- **FR-053** support production-oriented registry usage patterns.
- **FR-054** support promotion of governed AI assets across environments.
- **FR-055** support progression from local development workflows to development registries on cluster where applicable.
- **FR-056** support progression from development registries to production registries.
- **FR-057** preserve lineage and governance context during promotion.
- **FR-058** support inner-loop to outer-loop workflows.
- **FR-059** support use cases in which some users develop locally and push or promote governed asset metadata to shared environments.
- **FR-060** support use cases in which some users develop locally while interacting directly with development registries hosted on the cluster.
- **FR-061** support a clearer governed path from development to production for AI assets.

---

9. Security, Identity, and Access Control

- **FR-062** support controlled visibility of AI assets based on access permissions.
- **FR-063** support controlled management actions on AI assets based on access permissions.
- **FR-064** support controlled enablement of AI assets for use.
- **FR-065** support integration with enterprise identity and access systems.

- **FR-066** support authentication-aware workflows.
- **FR-067** support authorization-aware workflows.
- **FR-068** support RBAC-aligned access control expectations.
- **FR-069** support OAuth or OIDC-aligned identity integration expectations.
- **FR-070** support token-aware access patterns where cross-system access or promotion flows require them.
- **FR-071** support environment-aware access boundaries across development and production contexts.
- **FR-072** ensure that governed AI assets are manageable and usable only within the appropriate governance boundaries.

---

10. Policy Linkage and Enforcement Alignment

- **FR-073** support association of AI assets with policies, controls, or governance expectations that apply to them.
- **FR-074** support storage of policy references, policy metadata, or equivalent governance context associated with AI assets.
- **FR-075** support policy-aware visibility into what controls or expectations apply to an asset.
- **FR-076** support separation between registry-held governance and runtime enforcement.
- **FR-077** support alignment with external enforcement points such as gateways, policy engines, controllers, operators, or equivalent platform services.
- **FR-078** support policy-aware usage across development and production contexts.
- **FR-079** support future expansion toward stronger policy validation and enforcement integration without requiring redesign of the registry framework.

---

11. Gateway-Aware Platform Integration

- **FR-080** support gateway-aware integration for AI assets whose usage is mediated by gateway technologies.
- **FR-081** support association of governed AI assets with gateway-relevant metadata where applicable.
- **FR-082** support current gateway-oriented use cases relevant to inference, MCP, and semantic routing.

- **FR-083** preserve flexibility for future gateway-oriented use cases, including agent gateway patterns.
- **FR-084** support policy and governance linkage sufficient to align governed AI assets with gateway-based enforcement models.
- **FR-085** support runtime-reference awareness for assets such as MCP servers, agents, models, and guardrails when gateway alignment is relevant.

---

12. Catalog Interoperability

- **FR-086** preserve conceptual separation between registries and catalogs.
- **FR-087** support interoperability between registries and catalogs.
- **FR-088** support coherent user experience across registry and catalog interactions without collapsing the two into one concept.
- **FR-089** support use cases in which catalogs surface or reference governed assets from registries.
- **FR-090** support use cases in which governed registry information influences discoverability, curation, or approved availability in catalogs.

---

13. Auditability and Observability

- **FR-091** support auditability of governed AI assets and their management actions.
- **FR-092** support visibility into ownership, change history, lifecycle state, and governance status of AI assets.
- **FR-093** support audit context sufficient to understand who changed an asset and when.
- **FR-094** support observability-related metadata context for governed AI assets.
- **FR-095** support future integration with broader observability and operational tooling.
- **FR-096** support event-aware or change-aware behavior sufficient to enable downstream platform automation in the future.

---

14. OpenShift AI and Platform Alignment

- **FR-097** align with Red Hat OpenShift AI platform expectations.
- **FR-098** support Kubernetes-centered platform integration patterns.

- **FR-099** support tenancy-aware behavior suitable for shared platform environments.
- **FR-100** support namespaced or otherwise scoped governance contexts where appropriate.
- **FR-101** support OpenShift-native operational alignment where appropriate.
- **FR-102** support container-aware platform integration needs where applicable.
- **FR-103** support platform-native interaction models appropriate for OpenShift-based deployment and operation.

---

15. Future Growth and Ecosystem Readiness

- **FR-104** preserve the ability to add future AI asset types beyond the initial priority scope.
- **FR-105** preserve the ability to support future ecosystem onboarding, including third-party or partner-provided AI assets.
- **FR-106** preserve the ability to support future lifecycle automation.
- **FR-107** preserve the ability to support future evaluation-related integrations.
- **FR-108** preserve the ability to support future supply chain and trust-related capabilities.
- **FR-109** preserve the ability to support future policy, security, and gateway integration expansion.
- **FR-110** preserve the ability to support future platform capabilities that depend on governed AI asset metadata and relationships.

📏 Non-Functional Requirements

Non-Functional Requirements

Introduction

The following non-functional requirements define the quality attributes, platform characteristics, and operational expectations for the proposed AI Asset Registry solution. These requirements describe how the product should perform, scale, integrate, and operate across environments while supporting the functional requirements defined elsewhere in this document. They are written at the product level and do not prescribe a specific implementation design.

---

1. Extensibility and Maintainability

- **NFR-001** The solution should be designed to support long-term extensibility across AI asset types.
- **NFR-002** The solution should minimize the amount of core product change required to introduce support for a new AI asset type.
- **NFR-003** The solution should support maintainable extension patterns so that new asset types can be added without creating repeated one-off registry implementations.
- **NFR-004** The solution should preserve a consistent product experience even as new asset types and capabilities are added over time.
- **NFR-005** The solution should support independent evolution of asset-type-specific metadata and configuration needs without destabilizing the broader registry framework.
- **NFR-006** The solution should be maintainable across multiple releases and product phases as the number of supported asset types grows.

---

2. Scalability

- **NFR-007** The solution should scale to support growth in the number of governed AI assets, asset versions, and asset relationships.
- **NFR-008** The solution should scale to support multiple teams, projects, tenants, or namespaces operating concurrently.
- **NFR-009** The solution should scale without requiring a separate product experience for each supported AI asset type.
- **NFR-010** The solution should support growth in governance metadata, audit history, and relationship data without materially degrading core usability.
- **NFR-011** The solution should support future expansion to additional AI asset types, partner-provided assets, and broader ecosystem onboarding.

---

3. Performance and Responsiveness

- **NFR-012** The solution should provide responsive interaction for common registry operations such as asset lookup, metadata inspection, relationship viewing, and lifecycle status retrieval.
- **NFR-013** The solution should support efficient retrieval of governed asset metadata across supported interfaces.
- **NFR-014** The solution should support efficient handling of cross-asset relationship queries as usage expands.
- **NFR-015** The solution should avoid introducing unnecessary latency into common governance and discovery-adjacent workflows.
- **NFR-016** The solution should preserve acceptable responsiveness as adoption increases across teams, environments, and asset types.

---

4. Reliability and Availability

- **NFR-017** The solution should provide reliable access to governed AI asset metadata and governance state.
- **NFR-018** The solution should be designed for dependable operation across development and production-oriented environments.
- **NFR-019** The solution should minimize the risk that failures in one asset-specific extension or integration path disrupt the overall registry experience.
- **NFR-020** The solution should support continuity of governance information across environment progression and promotion workflows.
- **NFR-021** The solution should support recovery from operational failures without loss of authoritative governance metadata.

---

5. Security and Compliance Posture

- **NFR-022** The solution should align with enterprise security expectations for access control, identity integration, and governance boundaries.
- **NFR-023** The solution should support secure operation in environments that require RBAC, OAuth or OIDC, AuthN, AuthZ, and integration with enterprise identity providers.
- **NFR-024** The solution should support secure cross-system interactions where token-aware or delegated access patterns are required.
- **NFR-025** The solution should support separation of governance metadata from runtime enforcement responsibilities while preserving secure integration with enforcement layers.

- **NFR-026** The solution should be designed to support future compliance-oriented expansion such as stronger policy validation, trust metadata, and supply chain controls.

---

6. Auditability and Traceability

- **NFR-027** The solution should provide a strong audit trail for governed AI asset changes and governance actions.
- **NFR-028** The solution should support traceability of ownership, lifecycle transitions, approvals, and promotion history.
- **NFR-029** The solution should support traceability of relationships between governed AI assets.
- **NFR-030** The solution should support traceability across development and production contexts.
- **NFR-031** The solution should support downstream use of audit and change information for automation, operations, and governance review.

---

7. Observability

- **NFR-032** The solution should be observable in a way that supports platform operations, governance monitoring, and future operational automation.
- **NFR-033** The solution should provide sufficient metadata context to correlate registry actions with broader platform workflows and services.
- **NFR-034** The solution should support future integration with observability tooling used for logs, metrics, traces, and event-aware workflows.
- **NFR-035** The solution should support visibility into important operational events such as registration, update, approval, promotion, deprecation, and integration-related changes.
- **NFR-036** The solution should support future alignment with broader OpenShift and AI platform observability practices.

---

8. Interoperability and Integration

- **NFR-037** The solution should interoperate cleanly with adjacent platform capabilities such as catalogs, gateways, lifecycle automation, evaluation systems, and observability tooling.
- **NFR-038** The solution should support integration with OpenShift-native and Kubernetes-native platform patterns.

- **NFR-039** The solution should support integration with underlying registry technologies where needed without exposing those integrations as fragmented end-user product experiences.
- **NFR-040** The solution should support integration across APIs, CRD-oriented interfaces, UI workflows, CLI workflows, and automation paths.
- **NFR-041** The solution should preserve compatibility with future platform integrations without requiring major redesign.

---

9. OpenShift and Kubernetes Alignment

- **NFR-042** The solution should fit naturally within Red Hat OpenShift AI and broader OpenShift platform expectations.
- **NFR-043** The solution should support Kubernetes-centered operational and extensibility patterns where appropriate.
- **NFR-044** The solution should support multi-tenant and namespace-aware operational contexts.
- **NFR-045** The solution should support OpenShift-compatible deployment and operational expectations.
- **NFR-046** The solution should support platform-native integration approaches appropriate for OpenShift-based enterprise environments.

---

10. Usability and Consistency

- **NFR-047** The solution should provide a coherent and understandable product experience across supported interfaces and asset types.
- **NFR-048** The solution should minimize unnecessary complexity for users working with governed AI assets.
- **NFR-049** The solution should preserve conceptual clarity between registry responsibilities and catalog responsibilities.
- **NFR-050** The solution should support consistent interpretation of lifecycle, governance, and relationship concepts across asset types while still allowing appropriate flexibility.
- **NFR-051** The solution should support selective adoption by organizations without requiring all supported asset types to be adopted at once.

---

11. Portability and Environment Progression

- **NFR-052** The solution should support smooth progression from local and development-oriented workflows to more governed shared and production-oriented environments.
- **NFR-053** The solution should minimize friction in inner-loop to outer-loop workflows.
- **NFR-054** The solution should preserve lineage and governance continuity across promotion between environments.
- **NFR-055** The solution should support organizations that prefer local development with later promotion as well as organizations that prefer direct interaction with cluster-hosted development environments.
- **NFR-056** The solution should preserve portability of governance metadata and asset references across supported environments where applicable.

---

12. Evolvability and Future Readiness

- **NFR-057** The solution should be designed to evolve over time without requiring foundational redesign for each major new capability.
- **NFR-058** The solution should preserve the ability to support future lifecycle automation, evaluation integration, deeper gateway alignment, and expanded security and trust capabilities.
- **NFR-059** The solution should preserve the ability to support future ecosystem onboarding, including partner-provided and third-party AI assets.
- **NFR-060** The solution should preserve the ability to incorporate future trust, provenance, and supply chain requirements.
- **NFR-061** The solution should provide a durable foundation for future AI platform capabilities that depend on governed AI asset metadata, relationships, and lifecycle state.

🔐 Security and Access Requirements

Security and Access Requirements

This section defines the security and access expectations for the proposed AI Asset Registry solution. Because the registry is intended to govern important AI assets across development and production contexts, security must be treated as a foundational product concern rather than as a later add-on. The solution should support secure access, controlled enablement, policy-aware governance, and clean integration with broader platform enforcement layers such as gateways, policy engines, and identity systems.

Secure Visibility of AI Assets

Users need visibility into governed AI assets only when they are permitted to see them within the appropriate organizational, tenant, project, or environment boundaries.

The solution should support:

- controlled visibility of asset metadata based on access permissions
- separation of visibility across teams, projects, tenants, or namespaces where appropriate
- different visibility expectations across development and production contexts
- the ability to expose only the metadata and governance context appropriate for the requesting user or system

This is especially important because some assets may be broadly reusable, while others may be restricted due to ownership, environment, sensitivity, or governance state.

Controlled Management of AI Assets

Users need the ability to manage, update, approve, promote, enable, or retire AI assets only when they have the appropriate authority to perform those actions.

The solution should support:

- controlled creation and update of governed AI assets
- controlled approval and enablement workflows
- environment-aware restrictions on management actions
- stronger governance controls in production-oriented contexts than in development-oriented contexts

- separation of responsibilities where some users may consume assets while others govern them

Because organizations assign responsibilities differently, the product should support flexible but governed control over who can perform which actions.

Integration with Enterprise Identity and Access Systems

Users need the registry solution to fit cleanly into enterprise identity and access models rather than introducing a disconnected security experience.

The solution should support:

- integration with enterprise identity providers
- alignment with OAuth and OIDC-based access models
- support for authentication-aware and authorization-aware workflows
- alignment with role-based access control expectations
- secure identity-aware access across APIs, platform-native interfaces, and related user experiences

This is important both for OpenShift-native deployment patterns and for enterprise environments where identity, authentication, and authorization are centrally governed.

Role-Aware and Governance-Aware Access

Users need access decisions to account not only for identity, but also for governance context and lifecycle state.

The solution should support:

- role-aware access to asset metadata and management actions
- lifecycle-aware access and usage constraints
- the ability to distinguish between draft, approved, enabled, deprecated, or retired assets
- controlled enablement of assets for use
- governance-aware restrictions that vary based on environment or asset status

This helps ensure that assets are not only secured by identity, but also governed according to their current state and intended level of trust.

Policy-Aware Governance

Users need the registry to capture the policies, controls, and governance expectations that apply to AI assets, even when enforcement occurs elsewhere.

The solution should support:

- association of assets with applicable policies and controls
- clear visibility into what governance expectations apply to a given asset
- policy-aware metadata that can be understood by platform administrators and downstream systems
- the ability to carry governance intent across environments and promotion workflows
- a clean boundary between policy definition or association and runtime enforcement

This is important because the registry is intended to act as a governance system of record for AI assets, while gateways and policy engines may act as the runtime enforcement layer.

Separation Between Governance and Enforcement

Users need the solution to support a clean separation between recording governance intent and enforcing runtime controls.

The solution should support:

- registry-level storage of policy associations, governance state, and access expectations
- integration with external enforcement points such as gateways, policy engines, controllers, or operators
- the ability to align registry metadata with runtime enforcement systems without requiring the registry itself to be the enforcement engine
- future expansion toward stronger automation between governed registry state and runtime enforcement layers

This separation helps keep the registry focused on governed asset truth while allowing enforcement to happen in the runtime systems best suited for it.

Gateway-Aligned Security

Users need the registry to work cleanly with gateway-centered architecture, since many AI assets will be accessed, mediated, or protected through gateway technologies.

The solution should support:

- association of governed assets with gateway-relevant metadata where applicable
- support for assets whose access is mediated by inference gateways, MCP gateways, semantic routing layers, and future agent gateway patterns
- policy and governance linkage that can align with gateway-based enforcement
- awareness of runtime references, service endpoints, or related routing context when needed for governance
- future expansion toward deeper alignment with gateway-managed policy enforcement

This is particularly important for assets such as MCP servers, models, agents, and guardrails, where access and control may rely heavily on gateway patterns.

Secure Progression Across Environments

Users need AI assets to move from local or development-oriented contexts into more governed shared and production-oriented contexts without losing security controls or governance continuity.

The solution should support:

- environment-aware security boundaries
- secure promotion across environments
- preservation of governance context and lineage during promotion
- support for local-to-cluster and dev-to-prod workflows with appropriate access controls
- controlled use of shared development registries and production registries

Because some organizations will develop locally and later promote, while others will work directly against cluster-hosted development environments, the product should support both patterns securely.

Auditability of Security-Relevant Actions

Users need confidence that security-relevant changes and governance actions can be traced and reviewed.

The solution should support:

- auditability of asset creation, update, approval, enablement, promotion, deprecation, and related governance actions
- visibility into who made a change and when
- traceability of ownership, lifecycle transitions, and promotion history
- support for downstream operational review and governance oversight
- a stronger foundation for future compliance, operational monitoring, and automated controls

Auditability is especially important because the registry is intended to become foundational for broader platform governance.

Support for Future Trust and Compliance Expansion

Users need the solution to be secure in the present while also being ready for stronger future trust and compliance needs.

The solution should support:

- future expansion toward stronger policy validation
- future support for supply chain and trust metadata
- future support for provenance, attestation, and related governance evidence
- future alignment with deeper compliance-oriented and enterprise trust requirements
- security growth without requiring foundational redesign

Even if all trust and supply chain controls are not part of the initial scope, the platform should be designed so that those needs can be added over time.

Summary

Security and access for the AI Asset Registry solution should center on controlled visibility, governed management actions, enterprise identity integration, policy-aware governance, gateway alignment, secure promotion across environments, and strong auditability. The goal is not only to secure access to the registry itself, but to ensure that governed AI assets can participate safely and consistently in the broader OpenShift AI platform and future enterprise AI ecosystem.

🗃️ Catalog and Registry Interaction

Catalog and Registry Interaction

This section defines how catalogs and registries are expected to relate within the proposed AI Asset Registry product direction. The two concepts are intentionally separate and should remain separate. At the same time, they should work together in a way that feels coherent and useful to users. The goal is to preserve conceptual clarity while avoiding a fragmented product experience.

Distinct Responsibilities

Users need a clear distinction between the purpose of a registry and the purpose of a catalog so that governance and discoverability are not confused.

The solution should treat:

- **registries** as the system of governance, lifecycle, metadata, relationships, policy linkage, and operational context for AI assets
- **catalogs** as the system for discoverability, browsing, filtering, curation, and selection of AI assets

This separation is important because an asset may be governed in a registry without necessarily being broadly surfaced for discovery, and an asset may appear in a catalog because it has met the appropriate governance and readiness expectations.

A Coherent User Experience

Users should not have to understand internal product boundaries in order to work effectively with AI assets.

The solution should support:

- a coherent experience between catalog and registry interactions
- clear movement from discovery-oriented workflows into governance-aware asset details
- product behavior that feels connected even when catalog and registry responsibilities remain distinct
- reduced confusion about where users browse assets versus where those assets are governed

The goal is not to merge catalogs and registries into one concept, but to ensure they work together cleanly.

Catalogs Should Surface Governed Assets

Users need catalogs to present assets that can be understood and trusted within the broader platform.

The solution should support:

- catalogs surfacing assets that are governed through registries
- catalog experiences that reflect important registry context where appropriate
- the ability for catalog-visible assets to carry signals such as readiness, approval status, lifecycle relevance, or governance state
- curation and discoverability that can benefit from governed metadata

This helps ensure that discoverability is informed by governance rather than operating independently of it.

Registries Should Remain the Source of Governing Context

Users need a clear source of truth for lifecycle and governance information.

The solution should support:

- registries serving as the authoritative source for governed asset metadata and governance state
- registry-managed information informing how assets are surfaced, understood, or filtered in catalogs
- preservation of governance context even when assets are shown in a discovery-oriented catalog experience
- avoidance of duplicate or conflicting governance definitions across catalog and registry layers

This allows catalogs to benefit from registry information without taking over registry responsibilities.

Different Assets May Have Different Catalog Presence

Users need flexibility because not every governed asset should necessarily be surfaced in the same way.

The solution should support:

- some governed assets being surfaced broadly through catalogs
- some governed assets being available only in more limited or controlled discovery contexts
- governance state influencing whether an asset is appropriate for discovery or selection
- future flexibility for different curation strategies across asset types, environments, or organizations

This is important because some assets may be reusable and broadly relevant, while others may be highly specific, restricted, or not yet ready for wider consumption.

Catalogs Should Support Selection, Not Replace Governance

Users need catalogs to help them find and choose assets, but not to become the system responsible for governance.

The solution should support:

- catalogs helping users browse and select appropriate AI assets
- movement from catalog selection into richer governed asset details and relationships
- discovery-oriented actions that remain informed by registry-owned metadata
- a clean boundary in which catalogs do not replace lifecycle, approval, policy, or lineage responsibilities

This helps maintain the distinction between finding an asset and governing that asset.

Governance State Should Influence Discoverability

Users need discoverability to reflect the trust and readiness of governed assets.

The solution should support:

- governance state influencing whether and how an asset appears in catalog experiences

- visibility into whether an asset is draft, approved, enabled, deprecated, or otherwise governed
- curation and filtering informed by asset status and governance relevance
- future expansion toward more policy-aware or approval-aware catalog behavior

This ensures catalogs do not present assets as equally ready or trustworthy when governance indicates otherwise.

Support for Multiple Interaction Patterns

Users may interact with the platform in different ways depending on workflow, environment, or organizational maturity.

The solution should support:

- users who begin with catalog-based discovery and then move into governed asset details
- users who begin directly in registry-oriented workflows because they are managing assets rather than discovering them
- teams that rely heavily on catalogs for consumption
- teams that work primarily in registry and governance contexts

This flexibility is important because different organizations and teams will work differently.

Catalog and Registry Interaction Across Environments

Users need catalog and registry interaction to remain understandable across development and production contexts.

The solution should support:

- dev-oriented catalogs and registries where appropriate
- production-oriented catalogs and registries where appropriate
- governance state and promotion history informing what is surfaced in different environments
- a coherent path from discovery in one context to governed usage in another

This becomes especially important as assets move from development to more broadly trusted and production-ready states.

Foundation for Future Platform Workflows

Users need the relationship between catalogs and registries to support future platform growth.

The solution should support:

- future evaluation-aware discoverability
- future lifecycle-aware curation
- future alignment with gateways, policy systems, and other platform capabilities
- future ecosystem expansion without forcing a redesign of the catalog-registry boundary

This makes the separation strategically useful rather than just conceptually neat.

🔁 Dev to Prod / Inner to Outer Loop

Dev to Prod / Inner to Outer Loop

This section defines how the proposed AI Asset Registry solution should support the progression of AI assets from local and development-oriented workflows into more governed shared and production-oriented contexts. A major goal of this product opportunity is to make that progression easier, more understandable, and more governable across different organizational models. Some users will prefer local development with later promotion to the cluster, while others will work directly against shared development environments on the cluster. The product should support both patterns without forcing disconnected governance models.

Support Multiple Development Starting Points

Users need flexibility in how they begin development, because different teams and organizations will use different working models.

The solution should support:

- local development workflows where AI assets are created, refined, or iterated outside the cluster
- development workflows that interact directly with development-oriented registry capabilities hosted on the cluster
- the ability for different organizations to favor different entry points without breaking the overall governance model
- a consistent path forward regardless of whether work begins locally or directly in shared development environments

This is important because inner-loop workflows are not identical across customers, and the product should not force a single rigid model.

Provide a Governed Path from Development to Shared Environments

Users need a clear and governed way to move AI assets from early development into shared environments where assets can be reviewed, approved, reused, or prepared for production use.

The solution should support:

- progression from local or isolated development into shared development-oriented registry contexts
- registration of AI assets into governed development registries
- preservation of core metadata, ownership, and governance context as assets move into shared environments
- clearer visibility into where an asset is in its lifecycle and environment progression

This helps turn early AI work into governed platform assets rather than leaving it isolated in disconnected development workflows.

Distinguish Dev and Prod Registry Contexts

Users need development and production contexts to be clearly distinguishable because they serve different purposes and carry different governance expectations.

The solution should support:

- development-oriented registry contexts for active creation, iteration, experimentation, and early governance
- production-oriented registry contexts for more tightly governed, business-relevant, or operationally trusted assets
- different governance expectations between development and production environments
- continuity between dev and prod without treating them as interchangeable

This distinction is important because development environments should enable progress and experimentation, while production environments should reflect stronger governance, trust, and operational control.

Support Promotion Across Environments

Users need a governed promotion path that allows AI assets to move from one environment to another without losing identity, context, or trust.

The solution should support:

- promotion of governed AI assets across environments
- progression from development registries to production registries
- promotion-aware workflows that preserve the governed identity of an asset

- movement of AI asset metadata, references, relationships, and governance context in a controlled way
- stronger review or approval expectations as assets move toward production contexts

Promotion should not feel like starting over. It should preserve continuity while allowing the next environment to apply its own governance expectations.

Preserve Lineage Across the Full Lifecycle

Users need to understand where an AI asset came from, how it evolved, and how one environment-specific version relates to another.

The solution should support:

- lineage visibility across versions of an asset
- lineage visibility across environment progression
- preservation of identity as assets move from dev toward prod
- understanding of how promoted assets relate to earlier versions or earlier contexts
- a stronger foundation for downstream governance, evaluation, observability, and audit needs

This is important because promotion without lineage quickly makes governed AI assets difficult to trust and manage.

Preserve Governance Continuity During Promotion

Users need governance to travel with the asset rather than being re-created from scratch in each environment.

The solution should support:

- continuity of governance information as assets move between environments
- preservation of ownership, lifecycle context, and related governance metadata where appropriate
- retention of relationships and policy associations as part of progression workflows
- the ability for stricter production expectations to build on development governance rather than replace it entirely

This helps ensure that promotion is a governed progression rather than a disconnected handoff.

Support Different Organizational Operating Models

Users need the product to work for organizations with different development and operational preferences.

The solution should support:

- organizations that prefer local-first development with later push or promotion into cluster-hosted registries
- organizations that prefer direct interaction with cluster-hosted development environments from the start
- organizations with stronger separation between development and production governance
- organizations that adopt the platform incrementally across teams and asset types

This flexibility is important because customer environments and maturity levels vary significantly.

Enable Secure Progression Between Environments

Users need environment progression to remain secure as assets move from local or development contexts into more governed shared and production contexts.

The solution should support:

- environment-aware access boundaries
- secure promotion workflows
- security and governance controls that become stricter where appropriate as assets move toward production
- integration with access control, policy-aware governance, and approval expectations across environments
- preservation of auditability as assets move between contexts

This ensures that ease of progression does not come at the expense of governance or platform trust.

Reduce Friction in Inner-Loop to Outer-Loop Workflows

Users need the move from development to production to feel natural rather than cumbersome.

The solution should support:

- a smoother path from local iteration to governed shared environments
- a clearer path from development registries to production registries
- reduced duplication of work when moving assets forward
- fewer disconnected handoffs between experimentation, governance, and operational readiness
- progression that fits naturally into the broader OpenShift AI platform experience

This is strategically important because friction between inner-loop and outer-loop workflows often slows adoption and reduces the value of governance systems.

Support Future Automation of Environment Progression

Users need the product to be ready for deeper automation over time, even if full automation is not part of the initial scope.

The solution should support:

- future lifecycle automation tied to promotion and progression events
- future policy-aware checks during environment transitions
- future validation, evaluation, or trust checks as assets move toward production
- future operator-driven or platform-driven automation without requiring the basic product model to be redesigned

This is important because dev-to-prod progression is likely to become more automated as the surrounding platform matures.

Summary

The AI Asset Registry solution should provide a governed and flexible path from inner-loop development to outer-loop production usage. It should support both local-first and cluster-first development models, clearly distinguish dev and prod registry contexts, preserve lineage and governance continuity during promotion, and reduce friction as assets move across environments. The overall goal is to make AI asset progression easier to manage, easier to trust, and better aligned with long-term OpenShift AI platform needs.

The next strong tab would be **OpenShift AI and Platform Alignment**.

OpenShift AI and Platform Alignment

OpenShift AI and Platform Alignment

This section defines how the proposed AI Asset Registry solution should align with Red Hat OpenShift AI and the broader OpenShift platform. Because this product is intended to become a foundational capability for governing AI assets across development and production contexts, it should fit naturally into the platform rather than feeling like an isolated add-on. The solution should align with OpenShift-native patterns, support Kubernetes-centered workflows, and contribute to a broader platform model that includes catalogs, gateways, lifecycle automation, evaluation, observability, and governance.

Fit Naturally Within OpenShift AI

Users need the registry solution to feel like a natural part of the OpenShift AI ecosystem rather than a disconnected external capability.

The solution should support:

- alignment with Red Hat OpenShift AI product direction and platform expectations
- a cohesive experience alongside other AI platform capabilities
- compatibility with the way OpenShift AI organizes, governs, and exposes AI-related capabilities
- a product posture that treats governed AI assets as part of the broader platform ecosystem

This is important because the registry is intended to become an enabling layer for many other OpenShift AI capabilities over time.

Align with Kubernetes-Centered Platform Patterns

Users need the registry framework to work well in Kubernetes-based environments, especially where platform-native extension and automation patterns matter.

The solution should support:

- Kubernetes-centered interaction models
- platform-native extensibility patterns where appropriate
- a product experience that works well in cluster-based environments

- alignment with the operational expectations of teams managing Kubernetes and OpenShift platforms

This matters because OpenShift AI operates within a Kubernetes-based ecosystem, and the registry should align with that reality rather than ignoring it.

Support OpenShift-Native Interfaces

Users need the product to support interaction models that make sense in OpenShift-based enterprise environments.

The solution should support:

- API-based interaction for platform and external integrations
- Kubernetes-native interaction models such as CRD-oriented workflows where appropriate
- compatibility with UI, CLI, and automation workflows used across the platform
- a consistent product experience across these interfaces

This helps ensure the registry is usable by developers, platform teams, automation systems, and product surfaces without forcing a single narrow way of working.

Support Multi-Tenancy and Scoped Governance

Users need the registry to work in shared enterprise environments where teams, projects, business units, or customers may need different governance boundaries.

The solution should support:

- tenant-aware behavior
- namespace-aware or similarly scoped governance contexts
- separation of asset visibility and management across organizational boundaries
- flexibility for shared and isolated operating models within the same broader platform

This is especially important in OpenShift-based environments, where shared infrastructure often serves multiple teams and use cases at once.

Work Cleanly with OpenShift Security Models

Users need the registry to align with the security and access expectations already present in OpenShift and enterprise platform environments.

The solution should support:

- alignment with platform identity and access models
- compatibility with role-based access expectations
- secure behavior across shared cluster environments
- integration with broader security and governance workflows already present in the platform

This is important because the registry should reinforce platform security rather than introducing a parallel or inconsistent access model.

Support Container-Aware and Service-Aware AI Assets

Users need the registry to represent AI assets that may correspond to deployable services, running endpoints, artifacts, or externally referenced components.

The solution should support:

- metadata and references for assets that are deployed in containers where appropriate
- metadata and references for assets that point to services, runtimes, or externally hosted resources
- asset treatment that works for both deployable and non-deployable AI assets
- a broader understanding of AI assets as part of platform-managed workflows

This matters because OpenShift AI operates in a platform world where many governed assets are ultimately connected to running services or containerized capabilities, even when the registry itself remains metadata-first.

Align with Gateway-Centered Platform Direction

Users need the registry to fit within Red Hat's broader emphasis on gateways as enforcement and mediation layers for AI workloads.

The solution should support:

- alignment with inference gateway patterns
- alignment with MCP gateway patterns
- alignment with semantic routing patterns
- future alignment with agent gateway patterns
- governance metadata and policy context that can work with gateway-based enforcement models

This is especially important because the registry is intended to help govern assets whose runtime access and policy controls may be enforced through gateway technologies rather than directly within the registry itself.

Support Broader Platform Capabilities

Users need the registry to strengthen other important OpenShift AI platform capabilities rather than existing as a standalone island.

The solution should support alignment with:

- catalogs and discoverability experiences
- evaluation-related capabilities
- observability and operational monitoring
- lifecycle automation
- governance and policy workflows
- future supply chain and trust capabilities

This is important because the strategic value of the registry increases significantly when it acts as a shared foundation for adjacent platform services.

Preserve Compatibility with Existing and Transitional Platform Components

Users need the product to support current platform realities while still enabling long-term convergence.

The solution should support:

- coexistence with current platform capabilities that may already govern or expose certain asset types

- product evolution that does not require immediate consolidation of every asset type into one backend system
- a path for different asset domains to participate in one unified product experience over time
- alignment with current transitional needs while preserving long-term product direction

This is important because some asset domains already have established homes today, and the product should be able to work with that reality without compromising the broader vision.

Support Operational and Automation Readiness

Users need the registry to fit into the operational model of a modern OpenShift platform, including future automation and lifecycle coordination.

The solution should support:

- platform-friendly operational behavior
- automation-friendly interaction patterns
- future operator- or controller-driven workflows where appropriate
- future event-aware and lifecycle-aware platform actions tied to governed asset state

This is important because the registry is expected to become foundational for automation-rich platform capabilities over time.

Strengthen Red Hat's Platform Story

Users need this capability to contribute to a stronger and more cohesive product story for Red Hat OpenShift AI.

The solution should support:

- a clearer story for governing AI assets beyond models alone
- a more scalable product direction across multiple AI asset types
- stronger alignment between platform strategy and product capabilities
- a foundation that leadership, product teams, and engineering can build on over time

This is important because the registry is not only a feature opportunity, but also a strategic platform opportunity for Red Hat

⚠️ Risks

Risks

This section captures the key risks and considerations associated with the proposed AI Asset Registry solution. The purpose of this section is not to argue against the opportunity, but to ensure that important product and platform risks remain visible as the direction is reviewed by Product Management, Engineering, and Leadership. Because this proposal is intended to establish a foundational and extensible platform capability, it is important to acknowledge the main ways the effort could become fragmented, overly constrained, overly complicated, or misaligned with long-term goals.

Risk of One-Off Expansion

A major risk is that the platform continues to add registry capabilities one asset type at a time without a strong shared framework.

This could result in:

- duplicated product and engineering effort
- inconsistent user experiences across asset types
- fragmented governance models
- slower onboarding of future AI asset types
- reduced long-term maintainability

This is one of the primary risks the proposal is intended to address. If the product does not establish a durable extension model early, future asset coverage may become progressively harder to unify.

Risk of Over-Coupling to Current Backend Realities

There is value in working with current technologies and existing platform realities, but there is also a risk of letting near-term backend constraints define the long-term product model too rigidly.

This could result in:

- product direction being shaped too heavily by today's implementation constraints
- reduced flexibility for future asset types
- difficulty converging toward a more unified product experience over time
- a platform that feels permanently transitional rather than intentionally extensible

The proposal should preserve room for current realities while still protecting the broader product vision.

Risk of Over-Solutioning Too Early

Because this space is still evolving, there is a risk of prescribing specific technical solutions too early in the product definition process.

This could result in:

- locking engineering into overly narrow implementation assumptions
- limiting technical creativity and architecture evaluation
- turning a product requirements document into an architecture document prematurely
- making the proposal harder to align across a mixed audience of PMs, engineers, and leadership

This document is intended to define product expectations and desired behavior, not to prescribe the final engineering solution.

Risk of Blurring Catalog and Registry Responsibilities

If the distinction between catalogs and registries is not maintained, users may become confused about whether the solution is responsible for governance, discoverability, or both.

This could result in:

- conceptual confusion in the product
- duplicated or conflicting responsibilities
- weaker governance clarity
- reduced usability for teams trying to understand where to browse assets versus where to govern them

Preserving the conceptual separation while still supporting strong interaction between the two is important to product clarity.

Risk of Inconsistent Governance Across Asset Types

As more AI asset types are added, there is a risk that governance becomes inconsistent if each asset type evolves independently without enough shared principles.

This could result in:

- different lifecycle models that are difficult to understand
- inconsistent approval and promotion expectations
- varying policy treatment across assets
- reduced trust in the overall framework

The product should allow flexibility, but still preserve a coherent governance model across supported asset types.

Risk of Security and Policy Gaps

Because this solution sits close to governance, lifecycle, runtime access, and cross-system integration, weak alignment with security, identity, policy, and gateway enforcement models could undermine platform trust.

This could result in:

- unclear access boundaries
- weak alignment between governance state and runtime enforcement
- inconsistent policy handling across environments
- reduced confidence in enterprise readiness

Security and access expectations should remain foundational rather than treated as downstream enhancements.

Risk of Friction Between Development and Production

If the path from local or development workflows into governed shared and production contexts is too difficult, adoption may suffer.

This could result in:

- teams bypassing the registry process

- disconnected inner-loop and outer-loop workflows
- duplicated effort during promotion
- loss of lineage and governance continuity
- slower operational adoption of governed AI assets

A major product goal is to reduce this friction while still preserving strong governance.

Risk of Trying to Do Too Much in the Initial Scope

The long-term opportunity is broad, and there is a risk in trying to solve every future need in the initial phase.

This could result in:

- overly large MVP expectations
- slower delivery of foundational value
- unnecessary complexity in the first phase
- product sprawl before core patterns are proven

The initial direction should establish a durable foundation without requiring full maturity across every asset type or adjacent capability on day one.

❓ Open Questions

Open Questions

This section captures the key questions that should remain open for Product, Engineering, and Leadership discussion as the AI Asset Registry direction moves forward. These are not blockers to the proposal, but they are important areas for alignment as the product matures.

Unified Interface

Questions remain around how the unified experience should be anchored across API, Kubernetes-native, UI, CLI, and automation-based interactions, and which of those should be considered foundational versus layered experiences.

Plugin Model

Questions remain around how dynamic the plugin model should be in practice, how self-contained plugins should be, and how much asset-type-specific behavior should live in extensions versus the shared framework.

Policy Representation and Enforcement

Questions remain around how policy information should be represented in registry metadata, how governance intent should be expressed, and how that information should align with runtime enforcement systems such as gateways, policy engines, controllers, or operators.

Relationship Modeling

Questions remain around how deeply relationships between AI assets should be modeled in early phases, how configurable those relationships should be, and how much validation should occur in development versus production contexts.

Catalog and Registry Interaction

Questions remain around how registry state should influence discoverability, curation, selection, and asset availability in catalogs across different asset types and environments.

Dev and Prod Progression

Questions remain around how best to support different organizational operating models, especially where some teams prefer local-first workflows while others prefer direct interaction with cluster-hosted development registries.

Transitional and Long-Term Asset Placement

Questions remain around how current platform realities for different asset types should be handled over time while preserving the broader goal of one cohesive registry framework and experience.

Ecosystem Onboarding

Questions remain around how partner-provided or third-party assets should eventually be introduced into the registry model, including trust, validation, supportability, and governance expectations.

Future Growth

Future Growth

This section outlines the longer-term growth opportunities for the proposed AI Asset Registry solution. The purpose of this section is to make clear that the value of this product opportunity extends beyond the initial set of priority assets and beyond the first phase of delivery. While the near-term focus is on establishing a strong, extensible foundation, the broader opportunity is to create a durable governance layer for a growing ecosystem of AI assets across Red Hat OpenShift AI and related platform capabilities.

The long-term value of the solution increases if it can grow in a way that preserves the same core product principles established in this proposal:

- AI assets are treated as first-class citizens
- governance remains distinct from discoverability
- the registry framework stays metadata-first
- extensibility remains simple and sustainable
- the solution continues to feel like one cohesive product capability rather than a collection of disconnected point solutions

Expansion to Additional AI Asset Types

The most immediate future growth area is the expansion of registry support beyond the current priority asset types.

The solution should preserve the ability to grow into support for:

- notebooks
- pipelines
- evaluators
- skill packs
- code snippets
- workflows
- other future AI asset types that emerge as the ecosystem evolves

This is important because the long-term platform need is unlikely to stop with the initial registry types. The framework should make future expansion practical without requiring a new registry effort each time.

Broader Ecosystem Onboarding

Another major growth area is support for a broader ecosystem of externally sourced assets.

Over time, the solution should be able to grow toward support for:

- partner-provided AI assets
- third-party AI assets
- curated ecosystem contributions
- broader onboarding of governed external capabilities into the platform

This will become increasingly important for asset types such as MCP servers, models, guardrails, agents, and other reusable capabilities that may originate outside the organization.

Deeper Lifecycle Automation

As the surrounding platform matures, the registry can grow into a stronger lifecycle orchestration role.

Future growth may include:

- more automated progression across lifecycle states
- automation tied to promotion and environment transitions
- stronger coordination with lifecycle operators
- event-driven actions based on registry state changes
- richer governance workflows tied to approvals, validation, and retirement

This is especially important because registry value increases when governed state can drive broader platform behavior over time.

Stronger Evaluation and Quality Integration

The registry framework also creates an opportunity to support stronger evaluation-aware workflows.

Future growth may include:

- tighter linkage between governed assets and evaluators
- quality signals associated with registered assets
- evaluation-aware promotion decisions

- stronger validation before assets become broadly reusable or production-ready
- support for a more mature quality and trust model across AI assets

This would be particularly valuable for assets such as agents, prompts, models, guardrails, and skills.

Richer Observability and Operational Integration

As the platform matures, the registry can become more tightly integrated with observability and operational workflows.

Future growth may include:

- richer operational context associated with governed assets
- stronger eventing and downstream automation
- tighter correlation between asset state and platform operations
- broader visibility into how governed assets are used, promoted, and managed over time
- deeper integration with platform monitoring and operational tooling

This matters because the registry is expected to become foundational to broader platform governance and operations.

Stronger Policy and Enforcement Alignment

The registry can also grow into deeper alignment with policy systems and runtime enforcement layers.

Future growth may include:

- richer policy attachment models
- stronger policy validation before use or promotion
- tighter integration with gateways and policy engines
- more automated alignment between governed registry state and runtime enforcement
- broader support for policy-aware usage across environments and asset types

This is important because many AI assets will ultimately rely on other platform layers for runtime enforcement, and the registry should be able to grow with those needs.

Expanded Gateway Integration

Given Red Hat's strategic emphasis on gateways, gateway-related growth is especially important.

Future growth may include:

- deeper alignment with inference gateways
- deeper alignment with MCP gateways
- stronger support for semantic routing patterns
- future support for agent gateway models
- richer governance metadata that can inform gateway-based access, policy, and runtime behavior

This is particularly relevant for assets such as MCP servers, agents, models, and guardrails.

Trust, Provenance, and Supply Chain Growth

Over time, the registry framework should also be able to support stronger trust and supply chain capabilities.

Future growth may include:

- provenance metadata
- attestation and trust evidence
- support for stronger supply chain controls
- richer governance evidence tied to approval and promotion
- better support for enterprise trust requirements as the platform matures

Even if these capabilities are not required in full during the initial phase, the framework should preserve a path for them.

Broader Platform Integration

The registry has the potential to become a stronger shared foundation for multiple parts of the AI platform.

Future growth may include tighter integration with:

- catalogs
- evaluation systems

- observability tooling
- lifecycle automation
- gateway technologies
- governance and policy services
- future AI platform capabilities not yet fully defined

This is one of the strongest strategic arguments for treating the registry as a foundational platform capability rather than a narrow feature.

Long-Term Product Opportunity

The longer-term product opportunity is not only to support more assets, but to strengthen Red Hat's overall platform story.

Over time, this solution can help create:

- a more scalable governance model for AI assets
- a stronger product story beyond model-only registries
- better consistency across teams and asset types
- reduced fragmentation in future platform development
- a clearer foundation for enterprise AI governance across OpenShift AI